

Technical Design Brief - TaxonBodyMassML #032

Date: 02-28-2026 Team ID: #CS.032 Team Name: TaxonBodyMassML GitHub Repository: Our GitHub Repo	Team Members: - Grant Pasquantonio (pasquang@oregonstate.edu) - Hailey Prater (praterh@oregonstate.edu) - Anish Pendurti (pendurta@oregonstate.edu) Project Partner: Mark Novak
--	---

Change Log:

Date	Version	Sections Changed	Summary of Changes	Reasoning	Author(s)
2026-02-25	v1.1	All	Included figure captions on all images in the document	Feedback on v1.0	Everyone
2026-02-25	v1.1	Accessibility Notes	Included a note about the use of hyperlinks in the document	Repeated TA feedback on several past documents	Everyone
2026-03-01	v1.1	Interfaces/APIs and Data Contracts	Included “Interfaces and Owners” section	Updated Technical Design Brief rubric	Everyone
2026-03-01	v1.1	Consolidated Risk Register	Risks were reordered based on prioritization changes	Team Discussions of Priority	Everyone
2026-03-01	v1.1	Design Decisions and ADR Links	Altered Decision.03 to accurately reflect the added js and css files	Updates to the web-dev branch over time	Hailey
2026-03-01	v1.1	System context and high-level architecture	Changed decision tree name to XGBoost from	This is to match according changes to the model architecture.	Grant

System context and high-level architecture:

Our project is a web-based tool used to predict the body mass of one or multiple user-inputted species through the use of a machine learning predictive algorithm.

External actors:

- The End User is the primary external actor on the system. They may make requests for known or predicted masses through the web interface.
- Data entryist is another external actor on the system. As more measured species masses are discovered, they may be added to the initial database.
- Depending on the LLM prediction implementation, there may be an LLM API integration as an external actor.

Current components:

- Front-end web interface hosted on GitHub Pages
- Python Flask microservice which performs trivial species matching
- GitHub Actions CI pipeline

Future components:

- A XGBoosted Tree based species mass prediction service
- A LLM-based species mass prediction service

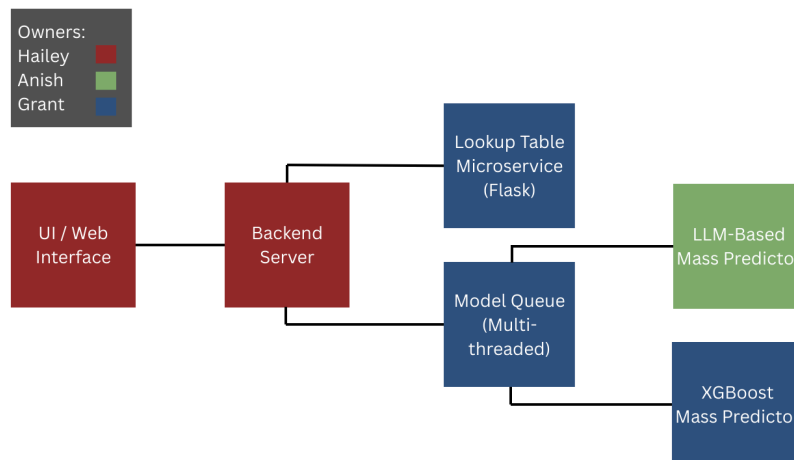


Fig.01: TaxonBodyMassML project architecture diagram

Connectors/protocols:

- We are using RESTful APIs for communication between system components.
- The Web Interface also incorporates communication over Hyper Text Transfer Protocol.

Boundaries:

- The front-end is a strictly static client and does not store data or directly run our machine learning models
- The UI is only responsible for direct interaction from the user (intaking user input and displaying output)
- All responsibility regarding data retrieval and prediction logic is placed on the backend
- Mass predictions only take place inside their respective microservice

Key Deployment Assumptions:

- We assume that the system hardware will be sufficient to handle expected traffic levels and run the prediction software as necessary.
- We assume that any third party software will maintain consistent API standards and functionality.
- We assume that the main elements of the system requirements will not shift significantly.

Interfaces/APIs and data contracts:

Interfaces and Owners:

- Species Look-Up API (Owned by Look-Up Table component; Used by Front-End component)
- Render Deployment Interface (Owned by Look-Up Table component)
- GitHub Pages Deployment Interface (Owned by Front-End component)

Contracts and examples:

HTTP Method: GET

URL (Endpoint):

https://haileystaxonbodymassml.onrender.com/single_species?species_name=<string>

```
1  {
2    "type": "object",
3    "properties": {
4      "species": { "type": "string" }
5    }
6  }
```

Fig.02: Look-up table request schema

```

1  {
2    "type": "object",
3    "properties": {
4      "domain": { "type": "string" },
5      "kingdom": { "type": "string" },
6      "phylum": { "type": "string" },
7      "class": { "type": "string" },
8      "order": { "type": "string" },
9      "family": { "type": "string" },
10     "genus": { "type": "string" },
11     "species": { "type": "string" },
12     "number_sources": { "type": "number" },
13     "source": { "type": "string" },
14     "mass_g": { "type": "number" }
15   }
16 }
17

```

Fig.03: Look-up table response schema

Example request:

GET https://haileystaxonbodymassml.onrender.com/single_species?species_name=natalus

Example response:

```

{
  "domain": "Animalia",
  "kingdom": "Mollusca",
  "phylum": "Cephalopoda",
  "class": "Nautiloidea",
  "order": "Nautilida",
  "family": "Nautilidae",
  "genus": "Nautilus",
  "species": "Nautilus macromphalus",
  "number_sources": 1,
  "source": "world-life-dataset-4",
  "mass_g": 450
}

```

HTTP Method: POST

URL (Endpoint): <https://haileystaxonbodymassml.onrender.com/predict>

Headers:

"Content-Type: application/json"

```

1  {
2    "type": "object",
3    "properties": {
4      "domain": { "type": "string" },
5      "kingdom": { "type": "string" },
6      "phylum": { "type": "string" },
7      "class": { "type": "string" },
8      "order": { "type": "string" },
9      "family": { "type": "string" },
10     "genus": { "type": "string" },
11     "species": { "type": "string" },
12     "newick_phylogeny": { "type": "string" }
13   }
14 }
15

```

Fig.04: Predictive component request schema

```

1  {
2    "type": "object",
3    "properties": {
4      "species": { "type": "string" },
5      "source": { "type": "string" },
6      "random_forest_mass_g": { "type": "number" },
7      "newick_mass_g": { "type": "number" },
8      "confidence_interval": { "type": "string" },
9      "tree_diagram": {
10        "type": "string",
11        "contentEncoding": "base64",
12        "contentType": "image/png"
13      }
14   }
15 }
16

```

Fig.05: Predictive component response schema

Example request:

POST <https://haileystaxonbodymassml.onrender.com/predict>

```

{
  "domain": "Animalia",
  "kingdom": "Mollusca",

```

```

"phylum": "Cephalopoda",
"class": "Nautiloidea",
"order": "Nautilida",
"family": "Nautilidae",
"genus": "Nautilus",
"species": "Nautilus macromphalus",
"newick_phylogeny": "(Nautilus macromphalus[0](Nautilus...",
}

```

Example response:

```

{
  "species": "Nautilus macromphalus",
  "source": "world-life-dataset-4",
  "random_forest_mass_g": 456.3472,
  "Newick_mass_g": "462.1385",
  "confidence_interval": "[443.2357, 481.0413]"
  "tree_diagram": "0erh4371..."
}
}
}

```

Error handling:

We will follow standard naming conventions for errors in communication:

- 404 Not Found: should only be used by the lookup table if it could not find a match for a given species.
- 500 Internal Server Error: may happen if the lookup table or predictive model are not responding correctly.

Typically, these will simply result in a failure of a given request. The user may try again, but if problems continue, then we will need to identify the issue and terminate it.

Authentication/Versioning Notes:

Since there is no need for the user to log in to the service, we do not need a comprehensive approach to Authentication.

Any version of the product before full release will be labeled using the 0.x.x template. After release of the product, versioning will start at 1.x.x. The first digit after 1 signifies major manual updates. The second digit after the 1 signifies automatic updates to the system as the database is correspondingly updated.

Individually-authored component sections:

Author: Grant Pasquantonio

Component: “Look-up Table”; Species taxonomy lookup table layer;

Purpose and Responsibilities

This component is responsible for gathering existing mass data and taxonomy data that is prerequisite to prediction.

- Loading the catalogue of life data with respect to species taxonomy
- Searching data for matching species or common name to input name
- Returning full taxonomy
- Also returning mass and source data if it exists

This information can be used to both directly inform the user if existing measured mass data exists. Otherwise, it helps gather the data necessary for a mass prediction.

Interfaces and Data Contracts

Species Lookup Flask Service

Created by: Grant

Used by: Hailey (front-end)

- This is the same interface as used in Haley Prater’s component section.
- Additional information is provided below:

HTTP Method: GET

URL (Endpoint):

https://haileystaxonbodymassml.onrender.com/single_species?species_name=<string>

```
1  {
2    "type": "object",
3    "properties": {
4      "species": { "type": "string" }
5    }
6  }
```

Fig.06: Schema for web interface server request

```

1  {
2    "type": "object",
3    "properties": {
4      "domain": { "type": "string" },
5      "kingdom": { "type": "string" },
6      "phylum": { "type": "string" },
7      "class": { "type": "string" },
8      "order": { "type": "string" },
9      "family": { "type": "string" },
10     "genus": { "type": "string" },
11     "species": { "type": "string" },
12     "number_sources": { "type": "number" },
13     "source": { "type": "string" },
14     "mass_g": { "type": "number" }
15   }
16 }
17

```

Fig.07: Schema for web interface server response

Server response will either be:

- 200 OK
- 404 Not Found

Example request:

GET https://haileystaxonbodymassml.onrender.com/single_species?species_name=natalus

Example response:

```

{
  "domain": "Animalia",
  "kingdom": "Mollusca",
  "phylum": "Cephalopoda",
  "class": "Nautiloidea",
  "order": "Nautilida",
  "family": "Nautilidae",
  "genus": "Nautilus",
  "species": "Nautilus macromphalus",
  "number_sources": 1,
  "source": "world-life-dataset-4",
  "mass_g": 450
}

```


Design Decisions

Reference **Decision.04** in the Design decisions and ADR links section.

Risks and Assumptions

Risk.04 - Lookup table dataset is too large for search-bar integration with the front-end. This could be a result of the massive dataset that will be loaded into the data frame we are handling the data with. It would likely work, but could potentially have a slow response time.

Mitigation: Make search apply only to a subset of species gathered via popularity data. We could gather this data using google trends.

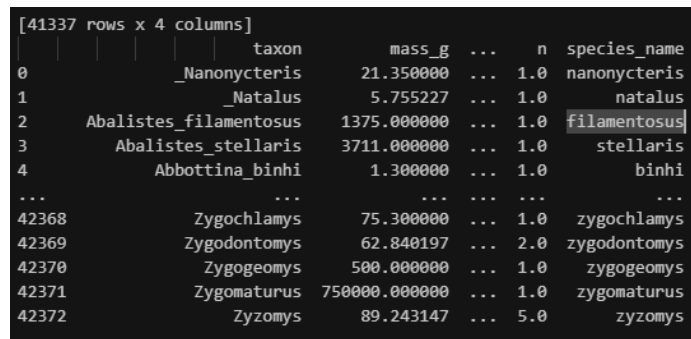
Assumptions: We assume that the taxonomic data will be complete for known and named species. We assume that the dataset will be updated as new species are discovered.

Dependencies

My component depends on the following:

- Flask for communication and running the microservice
- Pandas for data table manipulation
- Flask CORS for cross-origin resource sharing

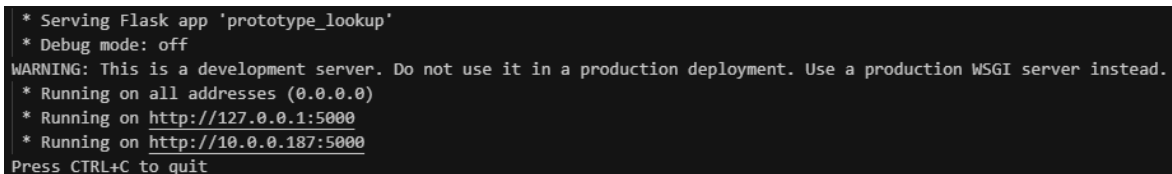
Evidence Artifacts



```
[41337 rows x 4 columns]
```

	taxon	mass_g	n	species_name
0	Nanonycteris	21.350000	1.0	nanonycteris
1	Natalus	5.755227	1.0	natalus
2	Abalistes_filamentosus	1375.000000	1.0	filamentosus
3	Abalistes_stellaris	3711.000000	1.0	stellaris
4	Abbottina_binhi	1.300000	1.0	binhi
...	...	...	...	...
42368	Zygochlamys	75.300000	1.0	zygochlamys
42369	Zygodontomys	62.840197	2.0	zygodontomys
42370	Zygoeomys	500.000000	1.0	zygoeomys
42371	Zygomaturus	750000.000000	1.0	zygomaturus
42372	Zyzomys	89.243147	5.0	zyzomys

Fig.08: Evidence of cleaning of taxon into species_name variable



```
* Serving Flask app 'prototype_lookup'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.0.0.187:5000
Press CTRL+C to quit
```

Fig.09: Microservice running as a Flask app

Author: Hailey Prater

Component: Front-end UI and web interaction layer; javascript integration

Purpose and Responsibilities

This component is responsible for all interaction with the user. Its main jobs include:

- Intaking a species name (or list of species names) from the user
- Sending an API request containing the species name(s) to the backend microservice
- Handling response from microservice and displaying to user (success or error)
- Providing an all-around accessible and easy-to-use web interface

In the future, these responsibilities will grow to include the random forest and LLM based predictive services, respectively.

Interfaces and Data Contracts

Species Lookup Flask Service

Created by: Grant

Used by: Hailey (front-end)

Endpoint:

GET https://haileystaxonbodymassml.onrender.com/single_species?species_name=<string>

Javascript Call:

```
const myLookupMicroservice = async (query) => {
  const url = `https://haileystaxonbodymassml.onrender.com/single_species?species_name=${encodeURIComponent(query)}`

  try {
    const response = await fetch(url)
    const data = await response.json()

    if (response.ok) {
      return {
        status: 'success',
        message: `${data.species_name} mass = ${data.mass_g} g`
      }
    } else {
      return { status: 'error', error: data.error || 'Unknown error' }
    }
  } catch (error) {
    console.error('Network error:', error)
    return { status: 'error', error: 'Network error' }
  }
}
```

Fig.10: Javascript call to render API

Example Responses:

```
▶ {mass_g: 5.75522663623422, species_name: 'Natalus'}
```

Fig.11: Success API response

```
▶ {error: "Missing 'species_name' parameter"}
```

Fig.12: Bad request response (status 400)

```
▶ {error: "Species 'asfsafasfasfas' not found"}
```

Fig.12: "Not Found" response (status 404)

Dependencies

My component depends on the following:

- HTML/CSS for UI layout and design
- JS for user interaction and element movements

- The prototype lookup microservice
- Render deployment
- Fetch requests
- GitHub Pages

Decisions

Refer to decision IDs 01, 02, and 03 in “Design decisions and ADR links” section below

Risks and Assumptions

Risk.01 - Render deployment fails after a resync

If render fails to deploy our Flask service after a push to the GitHub repo, it would remove all functionality from the web application. Instead of fetching species names and masses, it would simply return network errors. This would ruin any user’s experience with our site and limit trust.

Mitigation: We can monitor deployment previews, implement rollbacks, and log failure outputs.

Risk.02 - Updated CSS and JS are not prioritized by Pages

GitHub Pages sometimes serves cached versions of CSS and/or JS instead of what was most recently pushed, preventing users from seeing and interacting with the newest version of the UI.

Mitigation: We can update the version history of our CSS and JS files before pushing to GitHub.

Evidence Artifacts

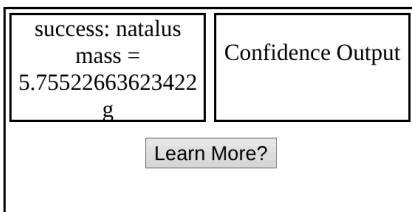


Fig.13: UI success after API call

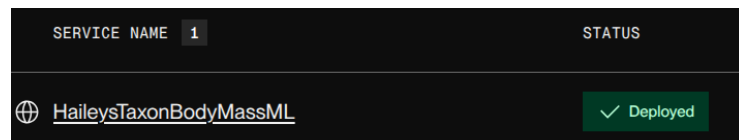


Fig.14: Successful render deployment status

Author: Anish Pendurti

Component: Data Merging

Purpose and Responsibilities

This component is responsible for gathering existing mass data and taxonomy data that is prerequisite to prediction.

- Loading the catalogue of life data with respect to species taxonomy dataset
- Searching data for any incorrect data formats or incorrect data and remove them from dataset
- Match species name with taxonomic information as well as mass of species and their phylum data

Interfaces and Data Contracts

This component is not connected to the rest of the system architecture via APIs or another method. It uses .csv data to interface with the rest of the system.

Design Decisions

Reference **Decision.04** in the Design decisions and ADR links section.

Risks and Assumptions

Risk.05 - Dataset is too large for merging and will cause errors like data loss and data duplication as well as unrequired/invalid data input.

Mitigation: Only copying data from the existing data from the open tree of life source data.

Assumptions: We assume that the taxonomic data will be complete for known and named species. We assume that the dataset will be updated as new species are discovered.

Dependencies

My component depends on the following:

- Open Tree of Life for taxonomy data
- Pandas for data merging
- Professor Novak for species mass data

Evidence Artifacts

```
PS C:\Users\pendu\Downloads\ac4054f8-c8a9-4a6e-ae39-8bff6c705318> python convert.py

df = pd.read_csv(
C:\Users\pendu\Downloads\ac4054f8-c8a9-4a6e-ae39-8bff6c705318\convert.py:5: ParserWarning: Skipping line 2300706: ' ' expected after ''

df = pd.read_csv(
C:\Users\pendu\Downloads\ac4054f8-c8a9-4a6e-ae39-8bff6c705318\convert.py:5: ParserWarning: Skipping line 2396859: ' ' expected after ''

df = pd.read_csv(
C:\Users\pendu\Downloads\ac4054f8-c8a9-4a6e-ae39-8bff6c705318\convert.py:5: ParserWarning: Skipping line 2403074: ' ' expected after ''

df = pd.read_csv(
C:\Users\pendu\Downloads\ac4054f8-c8a9-4a6e-ae39-8bff6c705318\convert.py:5: ParserWarning: Skipping line 2481571: ' ' expected after ''

df = pd.read_csv(
C:\Users\pendu\Downloads\ac4054f8-c8a9-4a6e-ae39-8bff6c705318\convert.py:5: ParserWarning: Skipping line 2538873: ' ' expected after ''

df = pd.read_csv(
C:\Users\pendu\Downloads\ac4054f8-c8a9-4a6e-ae39-8bff6c705318\convert.py:5: ParserWarning: Skipping line 2686721: ' ' expected after ''

df = pd.read_csv(
C:\Users\pendu\Downloads\ac4054f8-c8a9-4a6e-ae39-8bff6c705318\convert.py:5: ParserWarning: Skipping line 2750777: ' ' expected after ''

df = pd.read_csv(
C:\Users\pendu\Downloads\ac4054f8-c8a9-4a6e-ae39-8bff6c705318\convert.py:5: ParserWarning: Skipping line 2864682: ' ' expected after ''

df = pd.read_csv(
C:\Users\pendu\Downloads\ac4054f8-c8a9-4a6e-ae39-8bff6c705318\convert.py:5: ParserWarning: Skipping line 3012918: ' ' expected after ''
```

Fig.15: Figure caption (fill this in please)

Consolidated risk register (ranked by priority):

ID	Risk	Impact	Likelihood	Owner	Mitigation
R.01	Updated CSS styling and JS movements are not prioritized by GitHub Pages	High	High	Hailey	Update version history before pushing to GitHub; use consistent file paths
R.02	Look-up table dataset is too large for search-bar integration with the front-end.	Medium	High	Grant	Make search apply only to a subset of species gathered via popularity data. Could gather this data using google trends.
R.03	Services that are more complex than prototype_lookup may be too much for render's free tier	High	Medium	Primary: Grant Secondary: Hailey	Optimize predictive algorithm; precompute/preassign masses where possible
R.04	Microservice fails to deploy to render after syncing the forked repo	High	Medium	Hailey	Use render deployment previews; implement manual rollbacks; track build output/failure
R.05	Data Merging of the taxonomy data and species datasets	High	Medium	Anish	Run a search through database which deletes any data that does not follow the format given

Design decisions and ADR links:

Design Decisions and Rationale

Decision.01 - Deploying the prototype lookup service on Render

Owner - Hailey

We needed a simple deployment option for our Flask microservice that would allow it to run on our public-facing web page instead of locally from Grant's computer. This allowed each of us to continue testing the microservice without having to start it up manually. Render also returns deployment error messages that are easy to analyze and learn from. One concern we had about this decision was that render has limited compute for its free tier and a slow request time after prolonged inactivity, but these were outweighed by the simplicity of set-up and access.

ADR: [2025-11-12-praterh-ADR](#)

Git commit where decision was made: [push to web-dev branch](#)

Decision.02 - Using GitHub Pages for front-end hosting**Owner** - Hailey

We wanted to provide a public-facing host for our web application as soon as possible, and GitHub Pages was the best option for us. It is free, simple to implement, and supports our static HTML, CSS, and JS files. After syncing the forked repository from which our Pages is deployed, the updates are automatically applied to the public site; this makes it very easy to allow the whole team to see changes over time. There are lots of other sites for hosting web pages, but we decided on Pages to minimize the amount of new software we would have to learn. Pages runs off this forked repository (from web-dev branch): [TaxonBodyMassML Forked Repo](#)

Decision.03 - Keep CSS and JS in external files**Owner** - Hailey

I decided to create separate files for our CSS and JS ('style.css' and 'index.js' for the 'Home' page; 'info-pages.css' for the other tabs and 'help.js' for the 'Help' page; and 'tutorial.js' for the tutorial functionality) rather than keeping the code in-line or internal. Internal styling was something we considered, but ultimately, we wanted the styling and JS movements out of the HTML files and divided by functionality. This decision helps our code's interpretability and makes the linting process simpler. Having more files also results in less frequent and easier-to-fix merge conflicts.

Git commit where decision was made: [push to web-dev branch](#)

Decision.04 - Implement a lookup table as an independent microservice**Owner** - Grant

We needed a way to gather the taxonomic data for each species before running the mass prediction. This could either be implemented as an in-line program or an independent microservice. I chose to implement the lookup table as an independent microservice because it allows for more flexibility when incorporating features such as existing mass lookup and source reporting. It also is in line with the expectations discussed with the project partner.

[ADR](#)

The current implementation of this microservice is within the web-dev branch: [PR](#)

Traceability aid:

Risk ID	Requirement ID	Component Owner	ADR
R.04 (render redeploy fails)	REQ-001 and REQ-002	Hailey	 2025-11-12-praterh-ADR
R.02 (lookup table search integration fails)	REQ-007 and REQ-010	Grant	 2025-11-16-pasquang-ADR
R.05 (Data Merging of the taxonomy)	REQ-010	Anish	2025-11-16-pendurta-ADR

data and species datasets)			
----------------------------	--	--	--

Accessibility notes:

- Comprehensive Alt-text has been added to support the following images/figures:
 - High-Level Architecture Diagram
 - All schema images
- All included links are easily-readable hyperlinks